Realistic Mathematical Analysis of Modern Algorithms which Use SIMD Instructions for Better Performance

Supervisors: Cyril NICAUD
Pablo ROTONDO

Applicant: CHAU Dang Minh

Education: Maths and CS

MSc in Mathematics & Computer Science *Gustave Eiffel Univ., in progress*

- Relevant coursework: combinatorics, discrete optimization and complexity theory.

MSc in Applied Mathematics *Univ. of Limoges, 2025*

- Relevant coursework: stochastic optimization.

BEng in Computer Science *Univ. of Technology, Vietnam, 2024*

- Relevant coursework: data structures and algorithms, and compilers.

Research Experience: Analysis of Algorithms & Formal Methods

Compatibility between Open Automata and Session Types *M2 internship, Telecom Paris, 2025*

- **Core Result:** Proved that a simplified OA model is equivalent to multiparty session types, mapping safe composition to subtyping.
- **Takeaway for PhD:** Foundational experience in models of concurrent computing, useful for building models of SIMD parallelism.

Complexity Analysis of the Regex Matcher Hyperscan *M2 Internship, LIGM, in progress*

- **Core Analyses:** FDR string matcher (SIMD extension of Shift-Or), correctness of decomposition-based matching, and longest literal in average.
- **Takeaway for PhD:** Techniques for complexity analysis (from analytic combinatorics and probability theory), and experience with SIMD algorithms and their implementation.

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

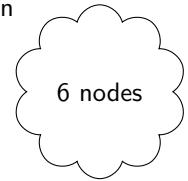
[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

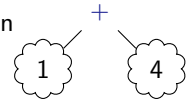
The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then



1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

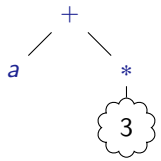
The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then



1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

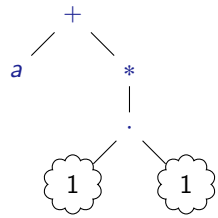
[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

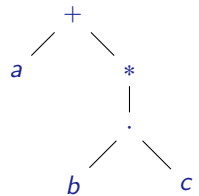
[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.



We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

Research Experience: Analysis of Algorithms - Details

Example of a regular expression (regex): $(a + b \cdot c) \cdot d \cdot e^*$, equivalent to $(a \cdot d + b \cdot c \cdot d)e^*$.

Hyperscan extracts literals ($a \cdot d$ and $b \cdot c \cdot d$ in the example) from a regex, then

1. matches all literals using FDR (or another string matcher [1]) *at once*,
2. verifies subregex matches using the previous results.

We assume a distribution over the regex trees, currently the BST-like [2].

Useful statistics to analyze asymptotically: the average of the number of literals, and the average longest length of the literals (on which we are concentrating).

The second statistic leads to analyzing the longest literal Y_n that a subtree *yields* to its parent.

[1] Qiu et al. (2021). Teddy: An efficient SIMD-based literal matching engine for scalable deep packet inspection.
[2] Binary Search Tree

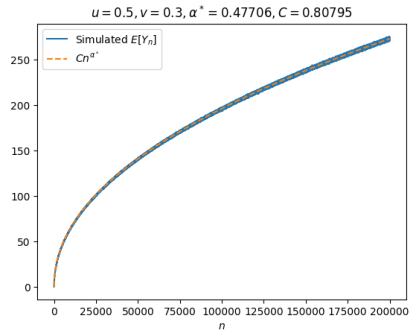
Research Experience: Analysis of Algorithms - Details

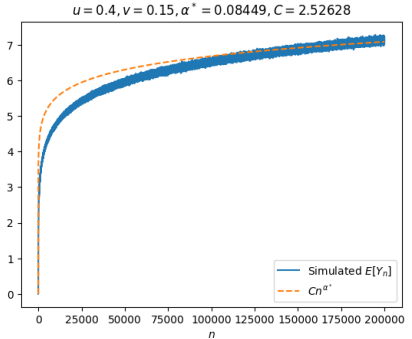
$$Y_{n+2} = \underbrace{I^{(n)}}_{\substack{\text{indicator of root} \\ \text{being concat.}}} \underbrace{(Y_{U_n} + Y_{n+1-U_n} + 1)}_{\substack{\text{take all of the subtrees,} \\ \text{plus the current}}} + \underbrace{J^{(n)}}_{\substack{\text{indicator of root} \\ \text{being alternation}}} \underbrace{\max(Y_{U_n}, Y_{n+1-U_n})}_{\substack{\text{take the longer} \\ \text{of the subtrees}}}, \quad Y_1 = Y_2 = 0.$$

For $\mathbb{E}[Y_n]$ when $\mathbb{E}[I^{(n)}] + \mathbb{E}[J^{(n)}] := u + v > 1/2$ (agreeing with real data),

Lower bound $\Omega(n^{\alpha})$ where α solves $\alpha + v2^{-\alpha} = 2u + v - 1$ | **Upper bound** $O(n^{2(u+v)-1})$

Conjecture: in some domain of (u, v), $\mathbb{E}[Y_n] \sim Cn^{\alpha^\star}$.





PhD Topic

Context and Motivation:

- Developers optimize their library algorithms in a fine-grained way e.g., they make assumptions on the input or leverage hardware features such as SIMD instructions.
- Researchers are interested in explaining if such optimizations are indeed efficient. Examples include both confirmation (TimSort [3]) and rejection (Lua's hybrid table [4])

Goal: Develop a framework for analyzing algorithms that use SIMD.

Fruitfulness:

- Direct applications to algorithm design.
- Requirements naturally lead to deep mathematical questions, e.g., $\mathbb{E}[Y_n]$ may not be analyzable by classical techniques.

[3] Auger, N., Jugé, V., Nicaud, C., & Pivoteau, C. (2018). On the worst-case complexity of TimSort.
[4] Martínez, C., Nicaud, C., & Rotondo, P. (2022). A probabilistic model revealing shortcomings in Lua's hybrid tables.

Thank you for attention !